

数组（Array）复习文档

目录

- 1. 数组常见处理方法总结
- 2. 双指针
 - 2.1 两数之和
 - 2.2 三数之和
 - 2.3 四数之和
 - 2.4 删除有序数组中的重复项
 - 2.5 删除有序数组中的重复项 II
 - 2.6 移动零
 - 2.7 移除元素
 - 2.8 有序数组的平方
 - 2.9 合并两个有序数组
 - 2.10 判断子序列
 - 2.11 颜色分类
 - 2.12 寻找重复数
- 3. 前缀和
 - 3.1 区域和检索 - 数组不可变
 - 3.2 二维区域和检索 - 矩阵不可变
 - 3.3 和为 K 的子数组
 - 3.4 和可被 K 整除的子数组
 - 3.5 寻找数组的中心下标
 - 3.6 最后 K 个数的乘积
 - 3.7 除自身以外数组的乘积
 - 3.8 连续的子数组和
 - 3.9 连续数组
 - 3.10 矩阵区域和
 - 3.11 表现良好的时间段
- 4. 差分数组
 - 4.1 区间加法
 - 4.2 拼车
 - 4.3 航班预订统计

- 5. 滑动窗口
 - 5.1 无重复字符的最长子串
 - 5.2 长度最小的子数组
 - 5.3 找到字符串中所有字母异位词
 - 5.4 最小覆盖子串
 - 5.5 字符串的排列
 - 5.6 乘积小于 K 的子数组
 - 5.7 存在重复元素 II
 - 5.8 存在重复元素 III
 - 5.9 替换后的最长重复字符
 - 5.10 最大连续1的个数 III
 - 5.11 将 x 减到 0 的最小操作数
 - 5.12 至少有 K 个重复字符的最长子串
 - 5.13 Rabin-Karp 字符串匹配
- 6. 二分搜索
 - 6.1 搜索插入位置
 - 6.2 搜索二维矩阵
 - 6.3 搜索二维矩阵 II
 - 6.4 搜索旋转排序数组
 - 6.5 搜索旋转排序数组 II
 - 6.6 寻找峰值
 - 6.7 山脉数组的峰顶索引
 - 6.8 找到 K 个最接近的元素
 - 6.9 爱吃香蕉的珂珂
 - 6.10 在 D 天内送达包裹的能力
 - 6.11 分割数组的最大值
 - 6.12 匹配子序列的单词数
 - 6.13 寻找旋转排序数组中的最小值
 - 6.14 查找和最小的 K 对数字
- 7. 延伸练习

1. 数组常见处理方法总结

技巧	核心思想	作用	典型应用
双指针 (同向)	快慢指针，一个遍历一个填充	原地去重、删除指定元素	删除排序数组重复项、移动零

技巧	核心思想	作用	典型应用
双指针 (相向)	左右指针向中间移动	N 数求和	两数之和、三数之和
前缀和	<code>preSum[i] = sum(nums[0..i])</code>	O(1) 时间求区间和/积	区域和检索、和为 K 的子数组
前缀和 + 哈希表	存前缀和到索引/次数的映射	求满足条件的子数组个数	和为 K 的子数组、连续数组
差分数组	记录相邻元素差值	快速对区间做加减法	航班预订统计、拼车
滑动窗口	维护 left/right 边界，动态伸缩窗口	求最长/最短连续子数组/子串	无重复字符最长子串、最小覆盖子串
二分搜索	每次排除一半搜索空间	有序数组快速查找	搜索插入位置、搜索旋转排序数组
二分答案 (值域二分)	对值做二分，判定可行性	求最小化最大值	珂珂吃香蕉、分割数组
桶思想	将元素按大小分配到桶中	快速判断值是否在范围内	存在重复元素 III

2. 双指针

2.1 两数之和

- 题目：1. 两数之和
- 描述：在数组中找出两个数，和为 target，返回它们的下标。
- 示例：
 - 输入： `nums = [2,7,11,15]` , `target = 9`
 - 输出： `[0,1]`

原始实现

```
class Solution:
    def twoSum(self, nums: List[int], target: int) -> List[int]:
        i, j = 0, len(nums) - 1
        temp = sorted(enumerate(nums), key=lambda x: x[1])
        while i < j:
            sum = temp[i][1] + temp[j][1]
            if sum > target:
                j -= 1
            elif sum < target:
                i += 1
            else:
                return [temp[i][0], temp[j][0]]
        return []
```

标准实现（哈希表法 — ★ 推荐）

评价：你的排序 + 相向双指针法可行，但破坏了原始下标。更通用的方法是 **哈希表**，一次遍历即可。

```
class Solution:
    def twoSum(self, nums: List[int], target: int) -> List[int]:
        seen = {}
        for i, num in enumerate(nums):
            complement = target - num
            if complement in seen:
                return [seen[complement], i]
            seen[num] = i
        return []
```

考试建议：哈希表法 $O(n)$ 一次遍历，无需排序，**强烈推荐**。排序双指针法适用于"返回数值而非下标"的场景（如三数之和）。

2.2 三数之和

- **题目：**15. 三数之和
- **描述：**找出数组中所有和为 0 且不重复的三元组。
- **示例：**
 - 输入：nums = [-1,0,1,2,-1,-4]

◦ 输出: $[[-1, -1, 2], [-1, 0, 1]]$

原始实现

```
class Solution:
    def threeSum(self, nums: list[int]) -> list[list[int]]:
        return self.threeSumTarget(nums, 0)

    def threeSumTarget(self, nums: List[int], target: int) -> List[List[int]]:
        nums.sort()
        res = []
        i = 0
        while i < len(nums):
            pairs = self.twoSumTarget(nums, i + 1, target - nums[i])
            for pair in pairs:
                pair.append(nums[i])
                res.append(pair)
            # 跳过重复元素
            for j in range(i + 1, len(nums)):
                if nums[j] == nums[i]:
                    i += 1
                else:
                    break
            i += 1
        return res

    def twoSumTarget(self, nums, start, target):
        lo, hi = start, len(nums) - 1
        res = []
        while lo < hi:
            sum = nums[lo] + nums[hi]
            if sum > target:
                hi -= 1
            elif sum < target:
                lo += 1
            else:
                res.append([nums[lo], nums[hi]])
                # 跳过重复
                for i in range(lo + 1, hi):
                    if nums[i] == nums[lo]:
                        lo += 1
                    else:
                        break
                for i in range(hi - 1, lo, -1):
                    if nums[i] == nums[hi]:
                        hi -= 1
```

```

        else:
            break
    lo += 1
    hi -= 1
return res

```

标准实现（★推荐）

评价：你的"泛化 nSumTarget"思路很好（可扩展为四数之和）。下面给出更紧凑的版本：

```

class Solution:
    def threeSum(self, nums: List[int]) -> List[List[int]]:
        nums.sort()
        res = []
        n = len(nums)
        for i in range(n):
            if i > 0 and nums[i] == nums[i - 1]:
                continue
            # 两数之和（相向双指针）
            lo, hi = i + 1, n - 1
            while lo < hi:
                total = nums[i] + nums[lo] + nums[hi]
                if total < 0:
                    lo += 1
                elif total > 0:
                    hi -= 1
                else:
                    res.append([nums[i], nums[lo], nums[hi]])
                    lo += 1
                    hi -= 1
                    while lo < hi and nums[lo] == nums[lo - 1]:
                        lo += 1
                    while lo < hi and nums[hi] == nums[hi + 1]:
                        hi -= 1
            return res

```

2.3 四数之和

- 题目：[18. 四数之和](#)
- 描述：找出数组中所有和为 target 且不重复的四元组。
- 示例：

- 输入: `nums = [1,0,-1,0,-2,2]` , `target = 0`
- 输出: `[[-2,-1,1,2],[-2,0,0,2],[-1,0,0,1]]`

原始实现

思路: 在 `threeSumTarget` 外再套一层循环, 与三数之和结构相同

```
class Solution:
    def fourSum(self, nums: List[int], target: int) -> List[List[int]]:
        nums.sort()
        res = []
        i = 0
        while i < len(nums):
            pairs = self.threeSumTarget(nums, i + 1, target - nums[i])
            for pair in pairs:
                pair.append(nums[i])
                res.append(pair)
            for j in range(i + 1, len(nums)):
                if nums[j] == nums[i]:
                    i += 1
                else:
                    break
            i += 1
        return res
# ... (threeSumTarget 和 twoSumTarget 同上)
```

标准实现

评价: 利用 `nSum` 递归泛化是经典思路。考试中如果只考四数之和, 可以直接在三数之和外套一层循环:


```

class Solution:
    def fourSum(self, nums: List[int], target: int) -> List[List[int]]:
        nums.sort()
        res = []
        n = len(nums)
        for i in range(n):
            if i > 0 and nums[i] == nums[i - 1]:
                continue
            for j in range(i + 1, n):
                if j > i + 1 and nums[j] == nums[j - 1]:
                    continue
                lo, hi = j + 1, n - 1
                while lo < hi:
                    total = nums[i] + nums[j] + nums[lo] + nums[hi]
                    if total < target:
                        lo += 1
                    elif total > target:
                        hi -= 1
                    else:
                        res.append([nums[i], nums[j], nums[lo], nums[hi]])
                        lo += 1
                        hi -= 1
                        while lo < hi and nums[lo] == nums[lo - 1]:
                            lo += 1
                        while lo < hi and nums[hi] == nums[hi + 1]:
                            hi -= 1
                return res

```

2.4 删除有序数组中的重复项

- **题目：**26. 删除有序数组中的重复项
- **描述：**原地删除有序数组中的重复项，使每个元素只出现一次，返回新长度。
- **示例：**
 - 输入：nums = [0,0,1,1,1,2,2,3,3,4]
 - 输出：5 , nums = [0,1,2,3,4,...]

原始实现

```
class Solution:
    def removeDuplicates(self, nums: List[int]) -> int:
        slow, fast = 0, 0
        target = -101
        ret = 0
        while fast < len(nums):
            if target != nums[fast]:
                nums[slow] = nums[fast]
                target = nums[fast]
                slow += 1
                ret += 1
            fast += 1
        return ret
```

标准实现 (★推荐)

评价：快慢指针，思路清晰。更简洁的写法：

```
class Solution:
    def removeDuplicates(self, nums: List[int]) -> int:
        if not nums:
            return 0
        slow = 0
        for fast in range(1, len(nums)):
            if nums[fast] != nums[slow]:
                slow += 1
                nums[slow] = nums[fast]
        return slow + 1
```

考试推荐：快慢指针是处理"原地修改"类问题的标配，**强烈推荐**。

2.5 删除有序数组中的重复项 II

- 题目：80. 删除有序数组中的重复项 II
- 描述：原地删除，使每个元素最多出现两次，返回新长度。
- 示例：
 - 输入：nums = [1,1,1,2,2,3]
 - 输出：5, nums = [1,1,2,2,3]

原始实现

```
class Solution:
    def removeDuplicates(self, nums: List[int]) -> int:
        slow, fast = 0, 0
        cur_num, num, ret = 10001, 1, 0
        while fast < len(nums):
            if cur_num != nums[fast]:
                cur_num = nums[fast]
                nums[slow] = cur_num
                slow += 1
                num = 1
                ret += 1
            else:
                if num == 1:
                    nums[slow] = cur_num
                    slow += 1
                    num = 1
                    ret += 1
                num += 1
            fast += 1
        return ret
```

标准实现 (★推荐)

评价：你的实现功能正确，但逻辑稍显复杂。一个更通用的技巧：因为只允许最多出现两次，直接比较 `nums[slow-2]` 与当前元素：

```
class Solution:
    def removeDuplicates(self, nums: List[int]) -> int:
        slow = 0
        for num in nums:
            if slow < 2 or num != nums[slow - 2]:
                nums[slow] = num
                slow += 1
        return slow
```

扩展：允许重复 `k` 次，只需将 `2` 改为 `k`。面试中非常实用的技巧。

2.6 移动零

- 题目: [283. 移动零](#)
- 描述: 将所有 0 移动到数组末尾, 保持非零元素相对顺序。
- 示例:
 - 输入: `nums = [0,1,0,3,12]`
 - 输出: `[1,3,12,0,0]`

原始实现

```
class Solution:
    def moveZeroes(self, nums: List[int]) -> None:
        slow, fast = 0, 0
        while fast < len(nums):
            if nums[fast] != 0:
                nums[slow], nums[fast] = nums[fast], nums[slow]
                slow += 1
            fast += 1
```

标准实现 (★ 推荐)

评价: 你的实现就是最优解! 快慢指针, 快指针遇非零就与慢指针交换。时间 $O(n)$, 空间 $O(1)$ 。

```
class Solution:
    def moveZeroes(self, nums: List[int]) -> None:
        slow = 0
        for fast in range(len(nums)):
            if nums[fast] != 0:
                nums[slow], nums[fast] = nums[fast], nums[slow]
                slow += 1
```

2.7 移除元素

- 题目: [27. 移除元素](#)
- 描述: 原地移除所有等于 `val` 的元素, 返回新长度。
- 示例:
 - 输入: `nums = [0,1,2,2,3,0,4,2]`, `val = 2`
 - 输出: `5`, `nums = [0,1,3,0,4]`

原始实现

```
class Solution:
    def removeElement(self, nums: List[int], val: int) -> int:
        slow, fast = 0, 0
        ret = 0
        while fast < len(nums):
            if nums[fast] != val:
                nums[slow] = nums[fast]
                slow += 1
                ret += 1
            fast += 1
        return ret
```

标准实现 (★推荐)

评价：快慢指针，标准解法。

```
class Solution:
    def removeElement(self, nums: List[int], val: int) -> int:
        slow = 0
        for fast in range(len(nums)):
            if nums[fast] != val:
                nums[slow] = nums[fast]
                slow += 1
        return slow
```

2.8 有序数组的平方

- 题目：977. 有序数组的平方
- 描述：对有序数组（含负数）的每个数求平方，结果也按非递减顺序排列。
- 示例：
 - 输入：nums = [-4, -1, 0, 3, 10]
 - 输出：[0, 1, 9, 16, 100]

原始实现

```
class Solution:
    def sortedSquares(self, nums: List[int]) -> List[int]:
        i, j = 0, len(nums) - 1
        p = len(nums) - 1
        ret = [0 for _ in range(len(nums))]
        while i <= j:
            if nums[i] ** 2 >= nums[j] ** 2:
                ret[p] = nums[i] ** 2
                i += 1
            else:
                ret[p] = nums[j] ** 2
                j -= 1
            p -= 1
        return ret
```

标准实现 (★推荐)

评价：相向双指针，这就是标准解法，最优美。时间 $O(n)$ ，空间 $O(1)$ （不计返回数组）。

```
class Solution:
    def sortedSquares(self, nums: List[int]) -> List[int]:
        n = len(nums)
        res = [0] * n
        i, j, k = 0, n - 1, n - 1
        while i <= j:
            if abs(nums[i]) > abs(nums[j]):
                res[k] = nums[i] * nums[i]
                i += 1
            else:
                res[k] = nums[j] * nums[j]
                j -= 1
            k -= 1
        return res
```

2.9 合并两个有序数组

- 题目：88. 合并两个有序数组
- 描述：将 `nums2` 合并进 `nums1` (`nums1` 空间足够大)，使结果有序。

- 示例:

- 输入: `nums1 = [1,2,3,0,0,0]` , `m = 3` , `nums2 = [2,5,6]` , `n = 3`
- 输出: `nums1 = [1,2,2,3,5,6]`

原始实现

```
class Solution:
    def merge(self, nums1: List[int], m: int, nums2: List[int], n: int) -> None:
        index_1, index_2 = m - 1, n - 1
        index = m + n - 1
        while index_1 >= 0 and index_2 >= 0:
            max_ = -1e10
            if nums1[index_1] >= nums2[index_2]:
                max_ = nums1[index_1]
                index_1 -= 1
            else:
                max_ = nums2[index_2]
                index_2 -= 1
            nums1[index] = max_
            index -= 1
        while index_1 >= 0:
            nums1[index] = nums1[index_1]
            index_1 -= 1
            index -= 1
        while index_2 >= 0:
            nums1[index] = nums2[index_2]
            index_2 -= 1
            index -= 1
```

标准实现 (★ 推荐)

评价: 从后往前填充是最优解法, 避免了从前填充需要移动元素的问题。简洁版本:

```

class Solution:
    def merge(self, nums1: List[int], m: int, nums2: List[int], n: int) -> None:
        p1, p2, p = m - 1, n - 1, m + n - 1
        while p1 >= 0 and p2 >= 0:
            if nums1[p1] > nums2[p2]:
                nums1[p] = nums1[p1]
                p1 -= 1
            else:
                nums1[p] = nums2[p2]
                p2 -= 1
            p -= 1
        # 只需处理 nums2 剩余 (因为 nums1 剩余已经在正确位置)
        nums1[:p2 + 1] = nums2[:p2 + 1]

```

2.10 判断子序列

- 题目: [392. 判断子序列](#)
- 描述: 判断字符串 s 是否是 t 的子序列。
- 示例:
 - 输入: s = "abc", t = "ahbgdc"
 - 输出: true

原始实现

```

class Solution:
    def isSubsequence(self, s: str, t: str) -> bool:
        i, j = 0, 0
        while i < len(s) and j < len(t):
            if s[i] == t[j]:
                i += 1
                j += 1
            else:
                j += 1
        return i == len(s)

```

标准实现 (★推荐)

评价: 你的双指针实现就是标准解法。时间 $O(m+n)$, 空间 $O(1)$ 。


```
class Solution:
    def isSubsequence(self, s: str, t: str) -> bool:
        i = 0
        for ch in t:
            if i < len(s) and ch == s[i]:
                i += 1
        return i == len(s)
```

2.11 颜色分类

- 题目：75. 颜色分类
- 描述：对只含 0、1、2 的数组原地排序（荷兰国旗问题）。
- 示例：
 - 输入：nums = [2,0,2,1,1,0]
 - 输出：[0,0,1,1,2,2]

原始实现

```
class Solution:
    def sortColors(self, nums: List[int]) -> None:
        len_ = max(nums) + 1
        count = [0 for _ in range(len_)]
        for num in nums:
            count[num] += 1
        i = -1
        for index in count:
            for _ in range(index):
                i += 1
                nums[i] = index
```

标准实现（三指针 — ★ 推荐）

评价：计数排序可以，但需要两趟遍历。荷兰国旗问题的经典解法是用 **三指针** 一趟搞定。

```
class Solution:
    def sortColors(self, nums: List[int]) -> None:
        zero, two = 0, len(nums) - 1
        i = 0
        while i <= two:
            if nums[i] == 0:
                nums[i], nums[zero] = nums[zero], nums[i]
                zero += 1
                i += 1
            elif nums[i] == 2:
                nums[i], nums[two] = nums[two], nums[i]
                two -= 1
            else:
                i += 1
```

考试建议：三指针 $O(n)$ 一趟遍历，面试中效果很好，**推荐理解**。如果时间紧张，计数排序也是一种选择。

2.12 寻找重复数

- 题目：287. 寻找重复数
- 描述：n+1 个数字在 [1, n] 范围，只有一个重复数，找出来（不能修改数组， $O(1)$ 空间）。
- 示例：
 - 输入：nums = [1,3,4,2,2]
 - 输出：2

原始实现

```
# 将数组视为链表: index -> nums[index]
class Solution:
    def next_node(self, nums, next):
        return nums[next]

    def next_next_node(self, nums, next):
        return nums[nums[next]]

    def findDuplicate(self, nums: List[int]) -> int:
        fast, slow = 0, 0
        while True:
            fast = self.next_next_node(nums, fast)
            slow = self.next_node(nums, slow)
            if slow == fast:
                break
        slow = 0
        while fast != slow:
            slow = self.next_node(nums, slow)
            fast = self.next_node(nums, fast)
        return slow
```

标准实现 (★ 推荐)

评价：你的"视为链表找环入口"思路非常巧妙！这是最优解，时间 $O(n)$ ，空间 $O(1)$ 。

```
class Solution:
    def findDuplicate(self, nums: List[int]) -> int:
        # Floyd 判圈
        slow = fast = nums[0]
        while True:
            slow = nums[slow]
            fast = nums[nums[fast]]
            if slow == fast:
                break
        slow = nums[0]
        while slow != fast:
            slow = nums[slow]
            fast = nums[fast]
        return slow
```

这道题还有二分法解法：在 $[1, n]$ 上二分，统计 $\leq \text{mid}$ 的数的个数，但不如 Floyd 法高效。

3. 前缀和

核心思想： $\text{preSum}[i] = \text{nums}[0] + \text{nums}[1] + \dots + \text{nums}[i-1]$ ，区间 $[l, r]$ 的和 = $\text{preSum}[r+1] - \text{preSum}[l]$ 。

可将 $O(n)$ 的区间查询优化为 $O(1)$ 。

3.1 区域和检索 - 数组不可变

- 题目：303. 区域和检索 - 数组不可变
- 描述：多次查询 `nums` 的区间和。
- 示例：
 - 输入： `nums = [-2,0,3,-5,2,-1]`，查询 `[0,2]`，`[2,5]`，`[0,5]`
 - 输出： `1`，`-1`，`-3`

原始实现

```
class NumArray:
    def __init__(self, nums: List[int]):
        self.preSum = [0 for _ in range(len(nums) + 1)]
        for i in range(1, len(nums) + 1):
            self.preSum[i] = self.preSum[i - 1] + nums[i-1]

    def sumRange(self, left: int, right: int) -> int:
        return self.preSum[right + 1] - self.preSum[left]
```

✓ 标准实现

评价：完全正确，这就是前缀和的标准模板。

```
class NumArray:
    def __init__(self, nums: List[int]):
        self.pre = [0]
        for num in nums:
            self.pre.append(self.pre[-1] + num)

    def sumRange(self, left: int, right: int) -> int:
        return self.pre[right + 1] - self.pre[left]
```

3.2 二维区域和检索 - 矩阵不可变

- 题目: [304. 二维区域和检索 - 矩阵不可变](#)
- 描述: 多次查询矩阵的子矩阵和。
- 示例:
 - 输入: `matrix = [[3,0,1,4,2],[5,6,3,2,1],[1,2,0,1,5],[4,1,0,1,7],[1,0,3,0,5]]`
 - 查询: `[2,1,4,3] → 8`

原始实现

```
class NumMatrix:
    def __init__(self, matrix: List[List[int]]):
        m, n = len(matrix) + 1, len(matrix[0]) + 1
        self.preSum = [[0 for _ in range(n)] for _ in range(m)]
        for i in range(1, m):
            for j in range(1, n):
                self.preSum[i][j] = self.preSum[i-1][j] + self.preSum[i][j-1] - self.preSum[i-1][j-1]

    def sumRegion(self, row1: int, col1: int, row2: int, col2: int) -> int:
        return (
            self.preSum[row2+1][col2+1]
            - self.preSum[row1][col2+1]
            - self.preSum[row2+1][col1]
            + self.preSum[row1][col1]
        )
```

✅ 标准实现

评价: 完全正确, 二维前缀和的标准实现。容斥原理: $\text{sum}(\text{大}) - \text{左} - \text{上} + \text{左上}$ 。

3.3 和为 K 的子数组

- 题目: [560. 和为 K 的子数组](#)
- 描述: 统计和为 K 的连续子数组的个数。
- 示例:
 - 输入: `nums = [1,1,1]`, `k = 2`
 - 输出: 2

原始实现

```
class Solution:
    def subarraySum(self, nums: List[int], k: int) -> int:
        n = len(nums) + 1
        preSum = [0] * n
        for i in range(1, n):
            preSum[i] = preSum[i - 1] + nums[i - 1]

        pos = {}
        for idx in range(n):
            val = preSum[idx]
            if val not in pos:
                pos[val] = []
            pos[val].append(idx)

        res = 0
        for i in range(1, n):
            need = preSum[i] - k
            if need in pos:
                cnt = bisect.bisect_left(pos[need], i)
                res += cnt
        return res
```

标准实现（前缀和 + 哈希表 — ★ 推荐）

评价：你用了"先建全表再二分"，时间复杂度 $O(n \log n)$ 。更优的是 **边遍历边计数** 的在线方法 $O(n)$ ：

```
class Solution:
    def subarraySum(self, nums: List[int], k: int) -> int:
        from collections import defaultdict
        pre_sum = 0
        count = defaultdict(int)
        count[0] = 1 # 初始：前缀和为 0 出现 1 次
        res = 0
        for num in nums:
            pre_sum += num
            # pre_sum - k 表示需要的前缀和
            res += count[pre_sum - k]
            count[pre_sum] += 1
        return res
```

核心： $\text{preSum}[j] - \text{preSum}[i] = k$ 等价于找前面有多少个 $\text{preSum}[j] - k$ 。边遍历边更新，**强烈推荐**。

3.4 和可被 K 整除的子数组

- 题目：974. 和可被 K 整除的子数组
- 描述：统计和能被 K 整除的连续子数组个数。
- 示例：
 - 输入：nums = [4,5,0,-2,-3,1] , k = 5
 - 输出：7

原始实现

```
class Solution:
    def subarraysDivByK(self, nums: List[int], k: int) -> int:
        n = len(nums) + 1
        preSum = [0 for _ in range(n)]
        for i in range(1, n):
            preSum[i] = preSum[i - 1] + nums[i - 1]

        dict = defaultdict(list)
        for i in range(n):
            dict[preSum[i] % k].append(i)

        res = 0
        for i in range(1, n):
            need = preSum[i] % k
            if need in dict:
                res += sum(1 for index in dict[need] if index < i)
        return res
```

标准实现（★推荐）

评价：与"和为 K 的子数组"类似，更优的是边遍历边计数：

```

class Solution:
    def subarraysDivByK(self, nums: List[int], k: int) -> int:
        from collections import defaultdict
        pre_sum = 0
        mod_count = defaultdict(int)
        mod_count[0] = 1
        res = 0
        for num in nums:
            pre_sum += num
            mod = pre_sum % k
            res += mod_count[mod]
            mod_count[mod] += 1
        return res

```

关键： $(\text{preSum}[j] - \text{preSum}[i]) \% k == 0$ 等价于 $\text{preSum}[j] \% k == \text{preSum}[i] \% k$ 。

3.5 寻找数组的中心下标

- 题目：724. 寻找数组的中心下标
- 描述：找到下标 i ，使得左侧元素和等于右侧元素和。
- 示例：
 - 输入：nums = [1,7,3,6,5,6]
 - 输出：3

原始实现

```

class Solution:
    def pivotIndex(self, nums: List[int]) -> int:
        n = len(nums) + 1
        preSum = [0 for _ in range(n)]
        for i in range(1, n):
            preSum[i] = preSum[i - 1] + nums[i - 1]
        for i in range(1, n - 1):
            leftSum = preSum[i - 1] - preSum[0]
            rightSum = preSum[n - 1] - preSum[i]
            if leftSum == rightSum:
                return i - 1
        return -1

```


标准实现 (★推荐)

评价：用前缀和即可，无需额外数组也可用"总和 + 左和"判断：

```
class Solution:
    def pivotIndex(self, nums: List[int]) -> int:
        total = sum(nums)
        left_sum = 0
        for i, num in enumerate(nums):
            if left_sum == total - left_sum - num:
                return i
            left_sum += num
        return -1
```

3.6 最后 K 个数的乘积

- 题目：1352. 最后 K 个数的乘积
- 描述：实现类 ProductOfNumbers，支持添加数字和查询最后 k 个数的乘积。
- 示例：
 - add(3)，add(0)，add(2)，add(5)，add(4)，getProduct(2) → 20，getProduct(3) → 40

原始实现

```
class ProductOfNumbers:
    def __init__(self):
        self.prefix = [1]

    def add(self, num: int) -> None:
        if num == 0:
            self.prefix = [1]
        else:
            self.prefix.append(self.prefix[-1] * num)

    def getProduct(self, k: int) -> int:
        if k >= len(self.prefix):
            return 0
        return self.prefix[-1] // self.prefix[len(self.prefix) - k - 1]
```

✅ 标准实现

评价：你的实现非常巧妙——遇到 0 就重置，利用前缀积的除法来算区间积。这是标准解法。

3.7 除自身以外数组的乘积

- 题目：238. 除自身以外数组的乘积
- 描述：返回数组 answer，其中 answer[i] 等于 nums 中除 nums[i] 之外其余各元素的乘积（不能用除法）。
- 示例：
 - 输入：nums = [1,2,3,4]
 - 输出：[24,12,8,6]

原始实现

```
class Solution:
    def productExceptSelf(self, nums: List[int]) -> List[int]:
        n = len(nums)
        prefix = [nums[0] for _ in range(n)]
        suffix = [nums[n - 1] for _ in range(n)]
        answer = [0 for _ in range(n)]
        for i in range(1, n):
            prefix[i] = prefix[i - 1] * nums[i]
            suffix[i] = suffix[i - 1] * nums[n - 1 - i]
        suffix.reverse()
        for i in range(n):
            if i - 1 >= 0 and i + 1 <= n - 1:
                answer[i] = prefix[i - 1] * suffix[i + 1]
            elif i - 1 < 0:
                answer[i] = suffix[i + 1]
            elif i + 1 > n - 1:
                answer[i] = prefix[i - 1]
        return answer
```

标准实现（★推荐）

评价：你的思路正确——前缀积 × 后缀积。但可以优化为 两次遍历 O(1) 额外空间：

```
class Solution:
    def productExceptSelf(self, nums: List[int]) -> List[int]:
        n = len(nums)
        answer = [1] * n

        # 从左到右：前缀积
        left = 1
        for i in range(n):
            answer[i] = left
            left *= nums[i]

        # 从右到左：后缀积
        right = 1
        for i in range(n - 1, -1, -1):
            answer[i] *= right
            right *= nums[i]

        return answer
```

考试推荐：一次前缀 + 一次后缀，巧妙利用 answer 数组存储中间结果，空间 $O(1)$ （不计输出）。

3.8 连续的子数组和

- **题目：**[523. 连续的子数组和](#)
- **描述：**判断是否存在长度至少为 2 的子数组，其和是 k 的倍数。
- **示例：**
 - 输入：nums = [23,2,4,6,6] , k = 7
 - 输出：true ([23,2,4,6,6] 和为 42，是 7 的倍数)

原始实现

```
class Solution:
    def checkSubarraySum(self, nums: List[int], k: int) -> bool:
        n = len(nums) + 1
        preSum = [0 for _ in range(n)]
        for i in range(1, n):
            preSum[i] = preSum[i - 1] + nums[i - 1]
        dict = {}
        for i in range(n):
            if preSum[i] % k not in dict:
                dict[preSum[i] % k] = i
        for i in range(1, n):
            need = preSum[i] % k
            if need in dict and i - dict[need] >= 2:
                return True
        return False
```

标准实现 (★推荐)

评价：你记录余数的**最早出现位置**是正确做法。但可以一次遍历完成：

```
class Solution:
    def checkSubarraySum(self, nums: List[int], k: int) -> bool:
        mod_map = {0: -1} # 余数 -> 最早索引
        pre_sum = 0
        for i, num in enumerate(nums):
            pre_sum += num
            mod = pre_sum % k
            if mod in mod_map:
                if i - mod_map[mod] >= 2:
                    return True
            else:
                mod_map[mod] = i # 只记录最早出现
        return False
```

3.9 连续数组

- 题目：[525. 连续数组](#)
- 描述：找到含有相同数量 0 和 1 的最长连续子数组。

- 示例:

- 输入: `nums = [0,1,0]`
- 输出: 2 (`[0,1]` 或 `[1,0]`)

原始实现

```
class Solution:
    def findMaxLength(self, nums: List[int]) -> int:
        n = len(nums) + 1
        preSum = [0 for _ in range(n)]
        for i in range(1, n):
            preSum[i] = preSum[i - 1] + (-1 if nums[i - 1] == 0 else 1)
        dict = {}
        res = 0
        for i in range(n):
            if preSum[i] not in dict:
                dict[preSum[i]] = i
            else:
                res = max(res, i - dict[preSum[i]])
        return res
```

✓ 标准实现

评价：完全正确！ 将 0 视为 -1、1 视为 +1，相同前缀和出现的位置之间区间和为 0（0 和 1 数量相等）。这是标准解法。

3.10 矩阵区域和

- 题目: [1314. 矩阵区域和](#)
- 描述: 对矩阵每个位置 (i,j) ，返回以 (i,j) 为中心、 $k \times k$ 为半径的矩形区域内所有元素之和。
- 示例:
 - 输入: `mat = [[1,2,3],[4,5,6],[7,8,9]]`, `k = 1`
 - 输出: `[[12,21,16],[27,45,33],[24,39,28]]`

原始实现

```
class Solution:
    def get_preSum(self, matrix):
        m, n = len(matrix) + 1, len(matrix[0]) + 1
        preSum = [[0 for _ in range(n)] for _ in range(m)]
        for i in range(1, m):
            for j in range(1, n):
                preSum[i][j] = preSum[i-1][j] + preSum[i][j-1] - preSum[i-1][j-1] + matrix[i-1][j-1]
        return preSum

    def sumRegion(self, preSum, row1, col1, row2, col2):
        return preSum[row2+1][col2+1] - preSum[row1][col2+1] - preSum[row2+1][col1] + preSum[row1][col1]

    def matrixBlockSum(self, mat: List[List[int]], k: int) -> List[List[int]]:
        preSum = self.get_preSum(mat)
        m, n = len(mat), len(mat[0])
        res = [[0 for _ in range(n)] for _ in range(m)]
        for i in range(m):
            for j in range(n):
                res[i][j] = self.getSum(preSum, i-k, j-k, i+k, j+k, m-1, n-1)
        return res
```

✅ 标准实现

评价：完全正确，二维前缀和的经典应用。注意边界处理即可。

3.11 表现良好的时间段

- 题目：1124. 表现良好的时间段
- 描述：若工作时长 > 8 小时记 +1，否则记 -1，要求和 > 0 的最长区间长度。
- 示例：
 - 输入：hours = [9,9,6,0,6,6,9]
 - 输出：3

原始实现

```
class Solution:
    def longestWPI(self, hours: List[int]) -> int:
        n = len(hours) + 1
        preSum = [0 for _ in range(n)]
        for i in range(1, n):
            preSum[i] = preSum[i-1] + (1 if hours[i-1] > 8 else -1)
        dict = defaultdict(list)
        for i in range(n):
            dict[preSum[i]].append(i)
        res = 0
        for i in range(1, n):
            need = preSum[i] - 1
            if preSum[i] > 0:
                res = max(res, i)
            if need in dict:
                res = max(res, i - dict[need][0])
        return res
```

标准实现（优化版）

评价：思路正确。优化：只保存每个前缀和的**第一次出现**，且只需查 `preSum[i] - 1`。

```
class Solution:
    def longestWPI(self, hours: List[int]) -> int:
        pre_sum = 0
        first_occur = {} # 前缀和 -> 第一次出现的索引
        res = 0
        for i, h in enumerate(hours):
            pre_sum += 1 if h > 8 else -1
            if pre_sum > 0:
                res = i + 1
            else:
                if pre_sum - 1 in first_occur:
                    res = max(res, i - first_occur[pre_sum - 1])
                if pre_sum not in first_occur:
                    first_occur[pre_sum] = i
        return res
```

4. 差分数组

核心思想： $\text{diff}[i] = \text{nums}[i] - \text{nums}[i-1]$ ，对区间 $[l, r]$ 加 val 只需 $\text{diff}[l] += \text{val}$ 、 $\text{diff}[r+1] -= \text{val}$ 。

可将 $O(n)$ 的区间加法优化为 $O(1)$ 。

4.1 区间加法

- **题目：** [370. 区间加法](#) (LeetCode VIP)
- **描述：** 给定一个长度为 length 的零数组，执行若干次区间加法操作。
- **示例：**
 - 输入： $\text{length} = 5$, $\text{updates} = [[1,3,2],[2,4,3],[0,2,-2]]$
 - 输出： $[-2,0,3,5,3]$

原始实现

```
class Solution:
    def increase(self, diff, start_index, end_index, inc):
        if start_index >= 0 and start_index < len(diff):
            diff[start_index] += inc
        if end_index + 1 < len(diff):
            diff[end_index + 1] -= inc

    def get_nums(self, diff):
        res = []
        res.append(diff[0])
        for i in range(1, len(diff)):
            res.append(diff[i] + res[i - 1])
        return res

    def getModifiedArray(self, length: int, updates: List[List[int]]) -> List[int]:
        diff = [0 for _ in range(length)]
        for update in updates:
            start_index, end_index, inc = update
            self.increase(diff, start_index, end_index, inc)
        return self.get_nums(diff)
```

✅ 标准实现

评价： 完全正确，差分数组的标准模板。


```

class Solution:
    def getModifiedArray(self, length: int, updates: List[List[int]]) -> List[int]:
        diff = [0] * length
        for l, r, val in updates:
            diff[l] += val
            if r + 1 < length:
                diff[r + 1] -= val

        res = [0] * length
        cur = 0
        for i in range(length):
            cur += diff[i]
            res[i] = cur
        return res

```

4.2 拼车

- 题目: [1094. 拼车](#)
- 描述: 判断车辆能否完成所有行程（容量固定，沿途上客下客）。
- 示例:
 - 输入: trips = [[2,1,5],[3,3,7]] , capacity = 4
 - 输出: false

原始实现

```

class Solution:
    def carPooling(self, trips: List[List[int]], capacity: int) -> bool:
        diff = [0 for _ in range(1000 + 1)]
        for trip in trips:
            inc, start_index, end_index = trip
            self.increase(diff, start_index, end_index, inc)
        return max(self.get_nums(diff)) <= capacity

```

✅ 标准实现

评价: 差分数组解决区间 + 容量问题，标准解法。

```

class Solution:
    def carPooling(self, trips: List[List[int]], capacity: int) -> bool:
        diff = [0] * 1001
        for num, start, end in trips:
            diff[start] += num
            diff[end] -= num # 注意: end 时已经下车了

        cur = 0
        for i in range(1001):
            cur += diff[i]
            if cur > capacity:
                return False
        return True

```

4.3 航班预订统计

- 题目: [1109. 航班预订统计](#)
- 描述: 有 n 个航班, 预订记录 $[i, j, k]$ 表示从 i 到 j 每个航班预订 k 个座位, 返回每个航班的总预订数。
- 示例:
 - 输入: `bookings = [[1,2,10],[2,3,20],[2,5,25]]`, `n = 5`
 - 输出: `[10,55,45,25,25]`

原始实现

```

class Solution:
    def corpFlightBookings(self, bookings: List[List[int]], n: int) -> List[int]:
        diff = [0 for i in range(n)]
        for booking in bookings:
            start_index, end_index, inc = booking
            self.increase(diff, start_index - 1, end_index - 1, inc)
        return self.get_nums(diff)

```

✅ 标准实现

评价: 完全正确。注意题目中航班从 1 开始编号, 要转成 0-index。

```

class Solution:
    def corpFlightBookings(self, bookings: List[List[int]], n: int) -> List[int]:
        diff = [0] * n
        for l, r, inc in bookings:
            diff[l - 1] += inc
            if r < n:
                diff[r] -= inc

        res = [0] * n
        cur = 0
        for i in range(n):
            cur += diff[i]
            res[i] = cur
        return res

```

5. 滑动窗口

核心框架：

```

left, right = 0, 0
while right < len(s):
    扩大窗口（移入 right 元素）
    right += 1
    while 需要收缩窗口:
        缩小窗口（移出 left 元素）
        left += 1
    更新答案

```

滑动窗口适用于求 **最长/最短/连续** 子数组/子串的问题。

5.1 无重复字符的最长子串

- 题目：3. 无重复字符的最长子串
- 描述：找不含重复字符的最长子串的长度。
- 示例：
 - 输入：s = "abcabcbb"
 - 输出：3 ("abc")

原始实现

```
class Solution:
    def lengthOfLongestSubstring(self, s: str) -> int:
        from collections import defaultdict
        window = defaultdict(int)
        left, right = 0, 0
        res = 0
        while right < len(s):
            chr = s[right]
            window[chr] += 1
            right += 1
            if window[chr] == 1:
                res = max(right - left, res)
            while left < right and window[chr] >= 2:
                chr_ = s[left]
                window[chr_] -= 1
                left += 1
        return res
```

标准实现 (★推荐)

评价：收缩条件只需看当前字符（新加入导致重复）：

```
class Solution:
    def lengthOfLongestSubstring(self, s: str) -> int:
        from collections import defaultdict
        window = defaultdict(int)
        left = res = 0
        for right, ch in enumerate(s):
            window[ch] += 1
            while window[ch] > 1:
                window[s[left]] -= 1
                left += 1
            res = max(res, right - left + 1)
        return res
```

5.2 长度最小的子数组

- 题目：[209. 长度最小的子数组](#)

- **描述：** 找和 $\geq$ target 的最短连续子数组的长度。
- **示例：**
 - 输入： target = 7 , nums = [2,3,1,2,4,3]
 - 输出： 2 ([4,3])

原始实现

```
class Solution:
    def minSubArrayLen(self, target: int, nums: List[int]) -> int:
        window = 0
        left, right = 0, 0
        res = 1e6
        while right < len(nums):
            num = nums[right]
            window += num
            right += 1
            while left < right and window >= target:
                res = min(right - left, res)
                num = nums[left]
                window -= num
                left += 1
        return res if res != 1e6 else 0
```

✅ 标准实现

评价：正确的滑动窗口实践。

```
class Solution:
    def minSubArrayLen(self, target: int, nums: List[int]) -> int:
        left = total = 0
        res = len(nums) + 1
        for right, num in enumerate(nums):
            total += num
            while total >= target:
                res = min(res, right - left + 1)
                total -= nums[left]
                left += 1
        return 0 if res > len(nums) else res
```

5.3 找到字符串中所有字母异位词

- 题目：438. 找到字符串中所有字母异位词
- 描述：在 s 中找到所有 p 的字母异位词的起始下标。
- 示例：
 - 输入：s = "cbaebabacd" , p = "abc"
 - 输出：[0,6]

原始实现

```
class Solution:
    def findAnagrams(self, s: str, p: str) -> List[int]:
        from collections import defaultdict
        window = defaultdict(int)
        need = defaultdict(int)
        for chr in p:
            need[chr] += 1
        left, right = 0, 0
        res = []
        while right < len(s):
            chr = s[right]
            window[chr] += 1
            right += 1
            while left < right and right - left == len(p):
                if self.is_right_substr(window, need):
                    res.append(left)
                chr = s[left]
                window[chr] -= 1
                if window[chr] == 0:
                    del window[chr]
                left += 1
        return res
```

标准实现 (★推荐)

评价：用字典比较即可。更高效的写法——字符频率数组：

```

class Solution:
    def findAnagrams(self, s: str, p: str) -> List[int]:
        need = [0] * 26
        for ch in p:
            need[ord(ch) - 97] += 1
        window = [0] * 26
        left = 0
        res = []
        for right, ch in enumerate(s):
            window[ord(ch) - 97] += 1
            if right - left + 1 == len(p):
                if window == need:
                    res.append(left)
                window[ord(s[left]) - 97] -= 1
                left += 1
        return res

```

5.4 最小覆盖子串

- 题目: [76. 最小覆盖子串](#)
- 描述: 在 s 中找到覆盖 t 中所有字符的最短子串。
- 示例:
 - 输入: s = "ADOBECODEBANC", t = "ABC"
 - 输出: "BANC"

原始实现

```
class Solution:
    def minWindow(self, s: str, t: str) -> str:
        from collections import defaultdict
        window = defaultdict(int)
        need = defaultdict(int)
        for chr in t:
            need[chr] += 1
        left, right = 0, 0
        res = [0, 0]
        while right < len(s):
            chr = s[right]
            window[chr] += 1
            right += 1
            while left < right and self.is_right_substr(window, need):
                res[:] = [left, right] if (right - left) < (res[1] - res[0]) or res == [0, 0] else
                chr = s[left]
                left += 1
                window[chr] -= 1
        return s[res[0]:res[1]]
```

标准实现 (★推荐)

评价：**match 变量**是性能关键——避免每次全表扫描。


```

class Solution:
    def minWindow(self, s: str, t: str) -> str:
        from collections import Counter
        need = Counter(t)
        window = {}
        left = match = 0
        start, min_len = 0, float('inf')

        for right, ch in enumerate(s):
            window[ch] = window.get(ch, 0) + 1
            if ch in need and window[ch] == need[ch]:
                match += 1

            while match == len(need):
                if right - left + 1 < min_len:
                    start = left
                    min_len = right - left + 1
                window[s[left]] -= 1
                if s[left] in need and window[s[left]] < need[s[left]]:
                    match -= 1
                left += 1

        return "" if min_len == float('inf') else s[start:start + min_len]

```

5.5 字符串的排列

- 题目: [567. 字符串的排列](#)
- 描述: 判断 s2 是否包含 s1 的排列。
- 示例:
 - 输入: s1 = "ab", s2 = "eidbaooo"
 - 输出: true ("ba")

原始实现

```
class Solution:
    def checkInclusion(self, s1: str, s2: str) -> bool:
        from collections import defaultdict
        window = defaultdict(int)
        need = defaultdict(int)
        for chr in s1:
            need[chr] += 1
        left, right = 0, 0
        while right < len(s2):
            chr = s2[right]
            window[chr] += 1
            right += 1
            while left < right and right - left == len(s1):
                if self.is_right_substr(window, need):
                    return True
                chr = s2[left]
                window[chr] -= 1
                if window[chr] == 0:
                    del window[chr]
                left += 1
        return False
```

标准实现 (★推荐)

```
class Solution:
    def checkInclusion(self, s1: str, s2: str) -> bool:
        need = [0] * 26
        window = [0] * 26
        for ch in s1:
            need[ord(ch) - 97] += 1
        left = 0
        for right, ch in enumerate(s2):
            window[ord(ch) - 97] += 1
            if right - left + 1 == len(s1):
                if window == need:
                    return True
                window[ord(s2[left]) - 97] -= 1
                left += 1
        return False
```

5.6 乘积小于 K 的子数组

- 题目: 713. 乘积小于 K 的子数组
- 描述: 统计所有乘积小于 k 的连续子数组个数。
- 示例:
 - 输入: nums = [10,5,2,6], k = 100
 - 输出: 8

原始实现

```
class Solution:
    def numSubarrayProductLessThanK(self, nums: List[int], k: int) -> int:
        window = 1
        left, right = 0, 0
        res = 0
        while right < len(nums):
            num = nums[right]
            window *= num
            right += 1
            while left < right and window >= k:
                num = nums[left]
                window = window // num
                left += 1
            res += right - left
        return res
```

✅ 标准实现

评价: 完全正确! res += right - left 是核心技巧。

```
class Solution:
    def numSubarrayProductLessThanK(self, nums: List[int], k: int) -> int:
        if k <= 1:
            return 0
        left = prod = res = 0
        for right, num in enumerate(nums):
            prod *= num
            while prod >= k:
                prod //= nums[left]
                left += 1
            res += right - left + 1
        return res
```

5.7 存在重复元素 II

- 题目: 219. 存在重复元素 II
- 描述: 判断是否存在距离 $\leq k$ 的两个相同元素。
- 示例:
 - 输入: `nums = [1,2,3,1]`, `k = 3`
 - 输出: `true`

原始实现

```
class Solution:
    def containsNearbyDuplicate(self, nums: List[int], k: int) -> bool:
        from collections import defaultdict
        window = defaultdict(int)
        left, right = 0, 0
        while right < len(nums):
            num = nums[right]
            window[num] += 1
            right += 1
            while left < right and right - left > k:
                num_ = nums[left]
                window[num_] -= 1
                left += 1
            if window[num] > 1:
                return True
        return False
```

✅ 标准实现

```
class Solution:
    def containsNearbyDuplicate(self, nums: List[int], k: int) -> bool:
        seen = {}
        for i, num in enumerate(nums):
            if num in seen and i - seen[num] <= k:
                return True
            seen[num] = i
        return False
```

5.8 存在重复元素 III

- 题目: [220. 存在重复元素 III](#)
- 描述: 判断是否存在下标差 $\leq \text{indexDiff}$ 且值差 $\leq \text{valueDiff}$ 的两个元素。
- 示例:
 - 输入: `nums = [1,2,3,1]`, `indexDiff = 3`, `valueDiff = 0`
 - 输出: `true`

原始实现

```
class Solution:
    def containsNearbyAlmostDuplicate(self, nums, indexDiff, valueDiff):
        width = valueDiff + 1
        left, right = 0, 0
        buckets = {}
        while right < len(nums):
            num = nums[right]
            bid = num // width
            if bid in buckets:
                return True
            if bid - 1 in buckets and abs(num - buckets[bid - 1]) <= valueDiff:
                return True
            if bid + 1 in buckets and abs(num - buckets[bid + 1]) <= valueDiff:
                return True
            buckets[bid] = num
            right += 1
            if right - left > indexDiff:
                num_left = nums[left]
                bid_left = num_left // width
                if bid_left in buckets and buckets[bid_left] == num_left:
                    del buckets[bid_left]
                left += 1
        return False
```

✅ 标准实现

评价: 桶思想标准解法。

```

class Solution:
    def containsNearbyAlmostDuplicate(self, nums: List[int], k: int, t: int) -> bool:
        if t < 0:
            return False
        buckets = {}
        width = t + 1
        for i, num in enumerate(nums):
            bid = num // width
            if bid in buckets:
                return True
            if bid - 1 in buckets and abs(num - buckets[bid - 1]) <= t:
                return True
            if bid + 1 in buckets and abs(num - buckets[bid + 1]) <= t:
                return True
            buckets[bid] = num
            if i >= k:
                del buckets[nums[i - k] // width]
        return False

```

5.9 替换后的最长重复字符

- 题目：424. 替换后的最长重复字符
- 描述：最多替换 k 个字符，使字符串中只有一种字符的最长子串长度。
- 示例：
 - 输入：s = "AABABBA", k = 1
 - 输出：4

原始实现

```
class Solution:
    def characterReplacement(self, s: str, k: int) -> int:
        from collections import defaultdict
        window = defaultdict(int)
        left, right = 0, 0
        res = 0
        while right < len(s):
            chr = s[right]
            window[chr] += 1
            right += 1
            while left < right and self.need_shrink(window, k):
                chr = s[left]
                window[chr] -= 1
                left += 1
            res = max(right - left, res)
        return res
```

标准实现 (★推荐)

评价：维护 maxFreq（窗口内出现最多的字符次数），窗口大小 - maxFreq > k 时收缩：

```
class Solution:
    def characterReplacement(self, s: str, k: int) -> int:
        from collections import defaultdict
        window = defaultdict(int)
        left = max_freq = res = 0
        for right, ch in enumerate(s):
            window[ch] += 1
            max_freq = max(max_freq, window[ch])
            while (right - left + 1) - max_freq > k:
                window[s[left]] -= 1
                left += 1
            res = max(res, right - left + 1)
        return res
```

5.10 最大连续1的个数 III

- 题目：[1004. 最大连续1的个数 III](#)

- **描述：**最多可以将 k 个 0 翻转为 1，求最长连续 1 的子数组长度。
- **示例：**
 - 输入：nums = [1,1,1,0,0,0,1,1,1,1,0] , k = 2
 - 输出：6

原始实现

```
class Solution:
    def longestOnes(self, nums: List[int], k: int) -> int:
        window = 0
        left, right = 0, 0
        res = 0
        while right < len(nums):
            num = nums[right]
            window += 0 if num == 1 else 1
            right += 1
            while left < right and window > k:
                num = nums[left]
                window -= 1 if num == 0 else 0
                left += 1
            res = max(res, right - left)
        return res
```

✓ 标准实现

```
class Solution:
    def longestOnes(self, nums: List[int], k: int) -> int:
        left = zeros = res = 0
        for right, num in enumerate(nums):
            if num == 0:
                zeros += 1
            while zeros > k:
                if nums[left] == 0:
                    zeros -= 1
                left += 1
            res = max(res, right - left + 1)
        return res
```


5.11 将 x 减到 0 的最小操作数

- 题目: 1658. 将 x 减到 0 的最小操作数
- 描述: 每次从数组最左或最右移除一个数, 求最小操作次数使移除数的和 = x。
- 示例:
 - 输入: nums = [1,1,4,2,3], x = 5
 - 输出: 2

原始实现

```
class Solution:
    def minOperations(self, nums: List[int], x: int) -> int:
        window = 0
        need = sum(nums) - x
        if need == 0:
            return len(nums)
        left, right = 0, 0
        res = [0, 0]
        while right < len(nums):
            num = nums[right]
            window += num
            right += 1
            while left < right and window >= need:
                if window == need:
                    res = [left, right] if right - left > res[1] - res[0] or res == [0, 0] else res
                    num = nums[left]
                    window -= num
                    left += 1
        return len(nums) - (res[1] - res[0]) if res != [0, 0] else -1
```

标准实现 (★推荐)

评价: 转为找和为 $\text{sum} - x$ 的最长连续子数组。

```

class Solution:
    def minOperations(self, nums: List[int], x: int) -> int:
        target = sum(nums) - x
        if target < 0:
            return -1
        if target == 0:
            return len(nums)

        left = cur_sum = max_len = 0
        for right, num in enumerate(nums):
            cur_sum += num
            while cur_sum > target:
                cur_sum -= nums[left]
                left += 1
            if cur_sum == target:
                max_len = max(max_len, right - left + 1)

        return -1 if max_len == 0 else len(nums) - max_len

```

5.12 至少有 K 个重复字符的最长子串

- 题目: [395. 至少有 K 个重复字符的最长子串](#)
- 描述: 找到每个字符出现次数 $\geq k$ 的最长子串长度。
- 示例:
 - 输入: $s = \text{"aaabb"}$, $k = 3$
 - 输出: 3 ("aaa")

原始实现

枚举字符种类数（1~26），配合滑动窗口

```
class Solution:
    def longestSubstring(self, s: str, k: int) -> int:
        return max([self.longestSubstring_with_n_chrs(s, k, i) for i in range(1, 26 + 1)])

    def longestSubstring_with_n_chrs(self, s, k, n):
        from collections import defaultdict
        window = defaultdict(int)
        left, right = 0, 0
        res = 0
        while right < len(s):
            chr = s[right]
            window[chr] += 1
            right += 1
            while left < right and len(window) > n:
                chr = s[left]
                window[chr] -= 1
                if window[chr] == 0:
                    del window[chr]
                left += 1
            if all(num >= k for num in window.values()):
                res = max(res, right - left)
        return res
```

✅ 标准实现

评价："限制字符种类数 + 滑动窗口"是正确解法之一。另一种常用的是分治法：

分治法

```
class Solution:
    def longestSubstring(self, s: str, k: int) -> int:
        if len(s) < k:
            return 0
        from collections import Counter
        count = Counter(s)
        for ch, num in count.items():
            if num < k:
                return max(self.longestSubstring(sub, k) for sub in s.split(ch))
        return len(s)
```

5.13 Rabin-Karp 字符串匹配

- **描述：**用滚动哈希在文本中匹配模式串，在 $O(n+m)$ 时间内完成。
- **示例：**
 - 输入：txt = "hello", pat = "ll"
 - 输出：2

原始实现

```
def rabinKarp(txt: str, pat: str) -> int:
    L = len(pat)
    R = 256
    Q = 1658598167
    RL = 1
    for i in range(1, L):
        RL = (RL * R) % Q

    patHash = 0
    for i in range(len(pat)):
        patHash = (R * patHash + ord(pat[i])) % Q

    windowHash = 0
    left, right = 0, 0
    while right < len(txt):
        windowHash = ((R * windowHash) % Q + ord(txt[right])) % Q
        right += 1
        if right - left == L:
            if windowHash == patHash:
                if pat == txt[left:right]:
                    return left
            windowHash = (windowHash - (ord(txt[left]) * RL) % Q + Q) % Q
            left += 1
    return -1
```

评价：Rabin-Karp 的核心思想是**滚动哈希**——利用前一个窗口的哈希值计算下一个窗口的哈希值，避免重复扫描。虽在实际中 Python 直接用 `find()` 即可，但滚动哈希的思想在解决某些复杂问题（如重复子串）时非常有用。

6. 二分搜索

核心模板（左闭右闭）：

```
def binary_search(nums, target):
    left, right = 0, len(nums) - 1
    while left <= right:
        mid = left + (right - left) // 2
        if nums[mid] == target:
            return mid
        elif nums[mid] < target:
            left = mid + 1
        else:
            right = mid - 1
    return -1
```

6.1 搜索插入位置

- 题目： [35. 搜索插入位置](#)
- 描述：在有序数组中找 target，若不存在返回它应该被插入的位置。
- 示例：
 - 输入： nums = [1,3,5,6] , target = 5 → 2
 - 输入： nums = [1,3,5,6] , target = 2 → 1

原始实现

```
class Solution:
    def searchInsert(self, nums: List[int], target: int) -> int:
        left, right = 0, len(nums) - 1
        while left <= right:
            mid = left + (right - left) // 2
            if nums[mid] < target:
                left = mid + 1
            elif nums[mid] == target:
                right = mid - 1
            elif nums[mid] > target:
                right = mid - 1
        return left
```

✅ 标准实现

评价：完全正确！等价于 `bisect.bisect_left`。当循环结束时，`left` 恰好是 $\geq \text{target}$ 的第一个位置。

6.2 搜索二维矩阵

- 题目：74. 搜索二维矩阵
- 描述：每行从左到右递增、下一行第一个大于上一行最后一个。判断 `target` 是否存在。
- 示例：
 - 输入：`matrix = [[1,3,5,7],[10,11,16,20],[23,30,34,60]]`，`target = 3`
 - 输出：`true`

原始实现

```
class Solution:
    def get(self, mat, index):
        col = len(mat[0])
        i, j = index // col, index % col
        return mat[i][j]

    def searchMatrix(self, matrix: List[List[int]], target: int) -> bool:
        left, right = 0, len(matrix) * len(matrix[0]) - 1
        while left <= right:
            mid = left + (right - left) // 2
            mid_num = self.get(matrix, mid)
            if mid_num < target:
                left = mid + 1
            elif mid_num == target:
                return True
            elif mid_num > target:
                right = mid - 1
        return False
```

✅ 标准实现

评价：将二维矩阵展平为一维进行二分，完全正确。

6.3 搜索二维矩阵 II

- 题目: [240. 搜索二维矩阵 II](#)
- 描述: 每行递增、每列递增, 判断 target 是否存在。
- 示例:
 - 输入:
matrix = [[1,4,7,11,15],[2,5,8,12,19],[3,6,9,16,22],[10,13,14,17,24],[18,21,23,26,30]] ,
target = 5
 - 输出: true

原始实现

```
class Solution:
    def searchMatrix(self, matrix: List[List[int]], target: int) -> bool:
        i, j = 0, len(matrix[0]) - 1
        while i < len(matrix) and j >= 0:
            num = matrix[i][j]
            if num == target:
                return True
            elif num > target:
                j -= 1
            elif num < target:
                i += 1
        return False
```

✅ 标准实现

评价: 从右上角开始搜索, 类似二叉搜索树。完全正确, 这是最优解!

6.4 搜索旋转排序数组

- 题目: [33. 搜索旋转排序数组](#)
- 描述: 旋转后的有序数组 (不含重复), 搜索 target 返回下标。
- 示例:
 - 输入: nums = [4,5,6,7,0,1,2] , target = 0
 - 输出: 4

原始实现

```
class Solution:
    def search(self, nums: List[int], target: int) -> int:
        import bisect
        left, right = 0, len(nums) - 1
        while left <= right:
            mid = left + (right - left) // 2
            if left == right:
                break
            if nums[mid] < nums[right]:
                right = mid
            elif nums[mid] > nums[right]:
                left = mid + 1
        index = left
        if target > nums[-1]:
            res = bisect.bisect_left(nums, target, 0, index)
        else:
            res = bisect.bisect_left(nums, target, index)
        return res if nums[res] == target else -1
```

标准实现 (★推荐)

评价：先找旋转点再二分思路正确。但一次二分更简洁：

```
class Solution:
    def search(self, nums: List[int], target: int) -> int:
        left, right = 0, len(nums) - 1
        while left <= right:
            mid = left + (right - left) // 2
            if nums[mid] == target:
                return mid
            if nums[left] <= nums[mid]: # 左半段有序
                if nums[left] <= target < nums[mid]:
                    right = mid - 1
            else:
                left = mid + 1
        else: # 右半段有序
            if nums[mid] < target <= nums[right]:
                left = mid + 1
            else:
                right = mid - 1
        return -1
```


6.5 搜索旋转排序数组 II

- 题目：81. 搜索旋转排序数组 II
- 描述：与上题相同，但数组可能包含重复元素。
- 示例：
 - 输入：nums = [2,5,6,0,0,1,2] , target = 0
 - 输出：true

原始实现

```
class Solution:
    def search(self, nums: List[int], target: int) -> bool:
        left, right = 0, len(nums) - 1
        while left <= right:
            while left + 1 < len(nums) and nums[left] == nums[left + 1]:
                left += 1
            while right - 1 >= 0 and nums[right] == nums[right - 1]:
                right -= 1
            mid = left + (right - left) // 2
            if nums[mid] == target:
                return True
            elif nums[mid] >= nums[left]:
                if nums[left] <= target <= nums[mid]:
                    right = mid - 1
                else:
                    left = mid + 1
            elif nums[mid] < nums[left]:
                if nums[mid] <= target <= nums[right]:
                    left = mid + 1
                else:
                    right = mid - 1
        return False
```

✓ 标准实现

评价：先去重再二分是正确的。更简洁的写法：遇到 `nums[left] == nums[mid]` 时直接 `left += 1`。

```
class Solution:
    def search(self, nums: List[int], target: int) -> bool:
        left, right = 0, len(nums) - 1
        while left <= right:
            mid = left + (right - left) // 2
            if nums[mid] == target:
                return True
            if nums[left] == nums[mid]:
                left += 1
                continue
            if nums[left] <= nums[mid]:
                if nums[left] <= target < nums[mid]:
                    right = mid - 1
                else:
                    left = mid + 1
            else:
                if nums[mid] < target <= nums[right]:
                    left = mid + 1
                else:
                    right = mid - 1
        return False
```

6.6 寻找峰值

- 题目: [162. 寻找峰值](#)
- 描述: 找数组中的一个峰值 ($\text{nums}[i] > \text{nums}[i+1]$ 且 $\text{nums}[i] > \text{nums}[i-1]$)。
- 示例:
 - 输入: `nums = [1,2,3,1]`
 - 输出: `2` (索引)

原始实现

```
class Solution:
    def findPeakElement(self, nums: List[int]) -> int:
        left, right = 0, len(nums) - 1
        while left <= right:
            mid = left + (right - left) // 2
            if left == right:
                return left
            if nums[mid] > nums[mid + 1]:
                right = mid
            elif nums[mid] < nums[mid + 1]:
                left = mid + 1
        return left
```

✅ 标准实现

```
class Solution:
    def findPeakElement(self, nums: List[int]) -> int:
        left, right = 0, len(nums) - 1
        while left < right:
            mid = (left + right) // 2
            if nums[mid] > nums[mid + 1]:
                right = mid
            else:
                left = mid + 1
        return left
```

6.7 山脉数组的峰顶索引

- 题目: [852. 山脉数组的峰顶索引](#)
- 描述: 数组先严格递增后严格递减, 返回峰顶索引。
- 示例:
 - 输入: arr = [0,10,5,2]
 - 输出: 1

原始实现

```
class Solution:
    def peakIndexInMountainArray(self, arr: List[int]) -> int:
        left, right = 0, len(arr) - 1
        while left <= right:
            mid = left + (right - left) // 2
            if left == right:
                return left
            if arr[mid] > arr[mid + 1]:
                right = mid
            elif arr[mid] < arr[mid + 1]:
                left = mid + 1
        return left
```

✅ 标准实现

评价：完全正确，与寻找峰值同理。

6.8 找到 K 个最接近的元素

- 题目：658. 找到 K 个最接近的元素
- 描述：在有序数组中找到离 x 最近的 k 个数。
- 示例：
 - 输入：arr = [1,2,3,4,5] , k = 4 , x = 3
 - 输出：[1,2,3,4]

原始实现

```
class Solution:
    def findClosestElements(self, arr: List[int], k: int, x: int) -> List[int]:
        import bisect
        p = bisect.bisect_left(arr, x)
        left = p - 1
        right = p
        res = []
        while left >= 0 and right < len(arr) and len(res) < k:
            if abs(arr[left] - x) <= abs(arr[right] - x):
                res.append(arr[left])
                left -= 1
            else:
                res.append(arr[right])
                right += 1
        while left >= 0 and len(res) < k:
            res.append(arr[left])
            left -= 1
        while right < len(arr) and len(res) < k:
            res.append(arr[right])
            right += 1
        res.sort()
        return res
```

标准实现 (★推荐)

评价：扩散法直观。更高效的二分法直接找区间左端点：

```
class Solution:
    def findClosestElements(self, arr: List[int], k: int, x: int) -> List[int]:
        left, right = 0, len(arr) - k
        while left < right:
            mid = (left + right) // 2
            if x - arr[mid] > arr[mid + k] - x:
                left = mid + 1
            else:
                right = mid
        return arr[left:left + k]
```

6.9 爱吃香蕉的珂珂

- 题目: [875. 爱吃香蕉的珂珂](#)
- 描述: 珂珂每小时最多吃 v 根香蕉, 求能吃完的最小速度 v 。
- 示例:
 - 输入: `piles = [3,6,7,11]`, `h = 8`
 - 输出: `4`

原始实现

```
class Solution:
    def minEatingSpeed(self, piles: List[int], h: int) -> int:
        left, right = 1, 1000000000
        while left <= right:
            mid = left + (right - left) // 2
            needed_time = self.get_time_required(piles, mid)
            if needed_time <= h:
                right = mid - 1
            else:
                left = mid + 1
        return left
```

✅ 标准实现

评价: 二分答案的标准模板。 `needed_time == h` 与 `< h` 合并处理。

```
class Solution:
    def minEatingSpeed(self, piles: List[int], h: int) -> int:
        def can_finish(v):
            return sum((p - 1) // v + 1 for p in piles) <= h
        left, right = 1, max(piles)
        while left < right:
            mid = (left + right) // 2
            if can_finish(mid):
                right = mid
            else:
                left = mid + 1
        return left
```

6.10 在 D 天内送达包裹的能力

- 题目: [1011. 在 D 天内送达包裹的能力](#)
- 描述: 求在 days 天内运完的最小运载能力。
- 示例:
 - 输入: weights = [1,2,3,4,5,6,7,8,9,10] , days = 5
 - 输出: 15

原始实现

```
class Solution:
    def shipWithinDays(self, weights: List[int], days: int) -> int:
        left, right = max(weights), 50000 * 500
        while left <= right:
            mid = left + (right - left) // 2
            needed_days = self.get_needed_days(weights, mid)
            if needed_days <= days:
                right = mid - 1
            else:
                left = mid + 1
        return left
```

✅ 标准实现

```
class Solution:
    def shipWithinDays(self, weights: List[int], days: int) -> int:
        def can_ship(cap):
            day = 1
            cur = 0
            for w in weights:
                if cur + w > cap:
                    day += 1
                    cur = 0
                cur += w
            return day <= days

        left, right = max(weights), sum(weights)
        while left < right:
            mid = (left + right) // 2
            if can_ship(mid):
                right = mid
            else:
                left = mid + 1
        return left
```

6.11 分割数组的最大值

- 题目: [410. 分割数组的最大值](#)
- 描述: 将数组分成 k 个连续子数组, 使各子数组和的最大值最小。
- 示例:
 - 输入: `nums = [7,2,5,10,8]` , `k = 2`
 - 输出: 18

原始实现

```
class Solution:
    def splitArray(self, nums: List[int], k: int) -> int:
        left, right = max(nums), sum(nums)
        if left == right:
            return left
        while left <= right:
            mid = left + (right - left) // 2
            needed_k = self.get_needed_k(nums, mid)
            if needed_k > k:
                left = mid + 1
            elif needed_k <= k:
                right = mid - 1
        return left
```

✅ 标准实现

评价：最小化最大值 = 二分答案的经典应用。

```
class Solution:
    def splitArray(self, nums: List[int], k: int) -> int:
        def can_split(max_sum):
            cnt = 1
            cur = 0
            for num in nums:
                if cur + num > max_sum:
                    cnt += 1
                    cur = 0
                cur += num
            return cnt <= k

        left, right = max(nums), sum(nums)
        while left < right:
            mid = (left + right) // 2
            if can_split(mid):
                right = mid
            else:
                left = mid + 1
        return left
```

6.12 匹配子序列的单词数

- 题目: 792. 匹配子序列的单词数
- 描述: 统计 words 中有多少是 s 的子序列。
- 示例:
 - 输入: s = "abcde" , words = ["a","bb","acd","ace"]
 - 输出: 3

原始实现

```
class Solution:
    def bi_search(self, indexs, index_target):
        left, right = 0, len(indexs) - 1
        while left <= right:
            mid = left + (right - left) // 2
            index = indexs[mid]
            if index < index_target:
                left = mid + 1
            elif index == index_target:
                right = mid - 1
            elif index > index_target:
                right = mid - 1
        return left if left != len(indexs) else -1

    def numMatchingSubseq(self, s: str, words: List[str]) -> int:
        from collections import defaultdict
        chr_dict = defaultdict(list)
        for index, chr in enumerate(s):
            chr_dict[chr].append(index)
        res = 0
        for word in words:
            res += 1 if self.match_one_word(word, chr_dict) else 0
        return res
```

✓ 标准实现

评价: 预处理 s 中每个字符出现的位置, 对每个单词二分查找匹配。

```
class Solution:
    def numMatchingSubseq(self, s: str, words: List[str]) -> int:
        from collections import defaultdict
        import bisect
        pos = defaultdict(list)
        for i, ch in enumerate(s):
            pos[ch].append(i)
        res = 0
        for word in words:
            prev = -1
            ok = True
            for ch in word:
                idx_list = pos[ch]
                j = bisect.bisect_right(idx_list, prev)
                if j == len(idx_list):
                    ok = False
                    break
                prev = idx_list[j]
            if ok:
                res += 1
        return res
```

6.13 寻找旋转排序数组中的最小值

- 题目: [153. 寻找旋转排序数组中的最小值](#)
- 描述: 在旋转后的有序数组（不含重复）中找最小值。
- 示例:
 - 输入: nums = [3,4,5,1,2]
 - 输出: 1

原始实现

```
class Solution:
    def findMin(self, nums: List[int]) -> int:
        left, right = 0, len(nums) - 1
        while left <= right:
            mid = left + (right - left) // 2
            if left == right:
                break
            if nums[mid] <= nums[right]:
                right = mid
            elif nums[mid] > nums[right]:
                left = mid + 1
        return nums[left]
```

✅ 标准实现

```
class Solution:
    def findMin(self, nums: List[int]) -> int:
        left, right = 0, len(nums) - 1
        while left < right:
            mid = (left + right) // 2
            if nums[mid] < nums[right]:
                right = mid
            else:
                left = mid + 1
        return nums[left]
```

6.14 查找和最小的 K 对数字

- 题目: [373. 查找和最小的 K 对数字](#)
- 描述: 从两个递增数组中各取一个数, 找和最小的 k 个数对。
- 示例:
 - 输入: `nums1 = [1,7,11]`, `nums2 = [2,4,6]`, `k = 3`
 - 输出: `[[1,2],[1,4],[1,6]]`

原始实现

```
class Solution:
    def kSmallestPairs(self, nums1: List[int], nums2: List[int], k: int) -> List[List[int]]:
        import heapq
        pq = []
        for num1 in nums1:
            heapq.heappush(pq, (num1 + nums2[0], num1, nums2[0], 0))
        ret = []
        for _ in range(k):
            _, num1, num2, i = heapq.heappop(pq)
            ret.append([num1, num2])
            if i + 1 >= len(nums2):
                continue
            heapq.heappush(pq, (num1 + nums2[i + 1], num1, nums2[i + 1], i + 1))
        return ret
```

✅ 标准实现

评价：最小堆 + 横向扩展是这道题的标准解法。

```
class Solution:
    def kSmallestPairs(self, nums1: List[int], nums2: List[int], k: int) -> List[List[int]]:
        if not nums1 or not nums2:
            return []
        import heapq
        heap = []
        for i in range(min(k, len(nums1))):
            heapq.heappush(heap, (nums1[i] + nums2[0], nums1[i], nums2[0], 0))
        res = []
        while heap and len(res) < k:
            _, x, y, j = heapq.heappop(heap)
            res.append([x, y])
            if j + 1 < len(nums2):
                heapq.heappush(heap, (x + nums2[j + 1], x, nums2[j + 1], j + 1))
        return res
```

7. 延伸练习

双指针延伸

题目	链接	技巧
11. 盛最多水的容器	https://leetcode.cn/problems/container-with-most-water	相向双指针
16. 最接近的三数之和	https://leetcode.cn/problems/3sum-closest	相向双指针
42. 接雨水	https://leetcode.cn/problems/trapping-rain-water	双指针 / 单调栈
611. 有效三角形的个数	https://leetcode.cn/problems/valid-triangle-number	相向双指针
15. 三数之和（进阶：允许重复）	https://leetcode.cn/problems/3sum	相向双指针 + 去重

前缀和延伸

题目	链接	技巧
1074. 元素和为目标值的子矩阵数量	https://leetcode.cn/problems/number-of-submatrices-that-sum-to-target	二维前缀和 + 哈希表
363. 矩形区域不超过 K 的最大数值和	https://leetcode.cn/problems/max-sum-of-rectangle-no-larger-than-k	二维前缀和
1442. 形成两个异或相等数组的三元组数目	https://leetcode.cn/problems/count-triplets-that-can-form-two-arrays-of-equal-xor	前缀异或

差分数组延伸

题目	链接	技巧
732. 我的日程安排表 III	https://leetcode.cn/problems/my-calendar-iii	差分数组
2406. 将区间分为最少组数	https://leetcode.cn/problems/divide-intervals-into-minimum-number-of-groups	差分数组 / 最小堆

滑动窗口延伸

题目	链接	技巧
30. 串联所有单词的子串	https://leetcode.cn/problems/substring-with-concatenation-of-all-words	滑动窗口 + 哈希表
239. 滑动窗口最大值	https://leetcode.cn/problems/sliding-window-maximum	单调队列
480. 滑动窗口中位数	https://leetcode.cn/problems/sliding-window-median	滑动窗口 + 堆
1423. 可获得的最大点数	https://leetcode.cn/problems/maximum-points-you-can-obtain-from-cards	滑动窗口 (反面思考)

二分搜索延伸

题目	链接	技巧
4. 寻找两个正序数组的中位数	https://leetcode.cn/problems/median-of-two-sorted-arrays	二分切割
69. x 的平方根	https://leetcode.cn/problems/sqrtx	二分答案
287. 寻找重复数	https://leetcode.cn/problems/find-the-duplicate-number	值域二分
668. 乘法表中第k小的数	https://leetcode.cn/problems/kth-smallest-number-in-multiplication-table	二分答案
719. 找出第 k 小的距离对	https://leetcode.cn/problems/find-k-th-smallest-pair-distance	二分答案 + 双指针
1482. 制作 m 束花所需的最少天数	https://leetcode.cn/problems/minimum-number-of-days-to-make-m-bouquets	二分答案

复习要点：

- **双指针**：同向（快慢）用于原地修改，相向用于 N 数求和
- **前缀和**：连加建表，区间查询 O(1)，配合哈希表可解子数组计数
- **差分数组**：区间加减 O(1)，逐项累加还原，适合批量区间操作
- **滑动窗口**：记住框架 + match 变量优化，最长/最短连续子串通用

- **二分搜索**：左闭右闭模板需牢记；值域二分用于"最小化最大值"类问题